



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

Vector Databases: Use Cases, Algorithms and Key Features

BACHELOR'S THESIS
- FIRST CYCLE OF STUDIES -

Student:
Kanita Kadušić

Mentor:
Prof. Dr. Haris Šupić

Sarajevo,
September 2025

Abstract

This thesis addresses the problem of approximate nearest neighbor (ANN) search in high-dimensional spaces, a core challenge in machine learning, computer vision, and natural language processing. Owing to the *curse of dimensionality*, exact search algorithms soon become inefficient, prompting the widespread adoption of approximate methods that provide faster retrieval while incurring a small reduction in accuracy.

The theoretical part of the thesis examines three dominant paradigms for ANN search: hash-based, tree-based, and graph-based methods. The hash-based approach is represented through Locality-Sensitive Hashing (LSH), which leverages families of hash functions to group similar vectors into the same buckets. The tree-based approach is illustrated by the Annoy library, which constructs multiple trees using random projections, balancing query efficiency, memory consumption, and retrieval accuracy. Particular emphasis is placed on the graph-based Hierarchical Navigable Small World (HNSW) model, which achieves logarithmic search complexity by combining a hierarchical structure with the properties of navigable small-world networks.

The practical part of the thesis focuses on examining vector management systems and tools. Milvus and Weaviate are presented as representative vector databases that support storage of embeddings, index construction, and similarity-based querying. FAISS, a standalone library providing high-performance ANN search methods, is introduced to illustrate algorithmic functionalities. Using embeddings generated from a selected dataset, the thesis demonstrates the process of importing embeddings, constructing indices, and performing search queries. In this way, the work connects theoretical concepts with practical implementation, providing an overview of the core algorithmic solutions and the software infrastructure enabling their application in real-world scenarios.

Keywords: high-dimensional data, approximate nearest neighbor (ANN), Hierarchical Navigable Small World (HNSW), vector databases, similarity search

Postavka zadatka završnog rada I ciklusa:
Vektorske baze podataka:
slučajevi upotrebe, algoritmi i ključne karakteristike

Cilj:

Cilj rada je istražiti teorijske osnove i praktične primjere primjene vektorskih baza podataka, te opis popularnih alata i sistema za upravljanje vektorskim bazama podataka.

Opis:

Tema završnog rada obrađuje vektorske baze podataka s naglaskom na njihovu primjenu u savremenim informacionim sistemima. U radu je potrebno opisati vektorsku reprezentaciju podataka. Posebnu pažnju treba posvetiti načinu indeksiranja i pretraživanja vektora unutar baza podataka, te razlikama u odnosu na klasične relacijske baze. U teorijskom dijelu prikazuju se osnovni algoritmi za pretraživanje po sličnosti, s naglaskom na metode *nearest neighbor* i *approximate nearest neighbor*. Nadalje, u praktičnom dijelu rada daje se pregled popularnih alata i sistema za upravljanje vektorskim bazama kao što su Weaviate, FAISS i Milvus, uključujući i opis konkretnih primjena vektorskih baza podataka. Mogući primjeri mogu uključivati npr. sisteme za preporučivanje, pretraživanje slika, *chatbot*-ove, sisteme za analizu dokumenata, ili neke druge samostalno odabrane primjere.

Očekivani rezultati:

- Opis teorijske podloge vektorskih baza podataka, uključujući njihove prednosti i razlike u odnosu na tradicionalne baze podataka.
- Opis osnovnih algoritama za pretraživanje po sličnosti, s naglaskom na metode *nearest neighbor* i *approximate nearest neighbor*.
- Prikaz primjera primjene vektorskih baza podataka.

Polazna literatura:

- [1] A. Gionis, P. Indyk, and R. Motwani, "Similarity Search in High Dimensions via Hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases*, September 1999, pp. 518-529.
- [2] N. Ukey, Z. Yang, B. Li, G. Zhang, Y. Hu, and W. Zhang, "Survey on Exact kNN Queries over High-Dimensional Data Space," *Sensors*, vol. 23, no. 2, p. 629, January 2023.
- [3] A. Guttman, "R-Trees: A Dynamic Index Structure for Spatial Searching," *ACM SIG-MOD Record*, vol. 14, no. 2, pp. 47-57, June 1984.

Red. prof. dr. Haris Šupić, dipl. ing. el.

Izjava o autentičnosti radova

Završni rad I ciklusa studija

Ime i prezime: Kanita Kadušić

Naslov rada: Vektorske baze podataka: slučajevi upotrebe, algoritmi i ključne karakteristike

Vrsta rada: Završni rad Prvog ciklusa studija

Broj stranica: 42

Potvrđujem:

- da sam pročitala dokumente koji se odnose na plagijarizam, kako je to definirano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja koja se odnose na I i II ciklus studija, integrirani studijski program I i II ciklusa i III ciklus studija na Univerzitetu u Sarajevu, kao i uputama o plagijarizmu navedenim na web stranici Univerziteta u Sarajevu;
- da sam svjesna univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem potpuno moj, samostalni rad, osim u dijelovima gdje je to naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili nekoj drugoj visokoškolskoj ustanovi;
- da sam jasno naznačila prisustvo citiranog ili parafraziranog materijala i da sam se referirala na sve izvore;
- da sam dosljedno navela korištene i citirane izvore ili bibliografiju po nekom od preporučenih stilova citiranja, sa navođenjem potpune reference koja obuhvata potpuni bibliografski opis korištenog i citiranog izvora;
- da sam odgovarajuće naznačila svaku pomoć koju sam dobila pored pomoći mentora i akademskih tutora/ica.

Sarajevo, septembar 2025.

Potpis:

Kanita Kadušić

Contents

List of Figures	vii
List of Tables	viii
List of Listings	ix
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Research Objectives	2
1.4 Methodology Overview	2
1.5 Structure of the Thesis	3
2 Theoretical Background	4
2.1 Unstructured Data	4
2.1.1 Embeddings	4
2.1.2 Distance Metrics	4
2.2 Nearest Neighbor Search	5
2.2.1 Exact Nearest Neighbor Search	5
2.2.2 Approximate Nearest Neighbor Search	6
3 Selected Algorithms	8
3.1 Locality-Sensitive Hashing (LSH)	8
3.1.1 Introduction to LSH	8
3.1.2 Description of LSH	8
3.1.3 Analysis of LSH	10
3.2 Approximate Nearest Neighbors Oh Yeah (Annoy)	12
3.2.1 Introduction to Annoy	12
3.2.2 Description of Annoy	12
3.2.3 Analysis of Annoy	17
3.3 Hierarchical Navigable Small World (HNSW)	18
3.3.1 Introduction to HNSW	18
3.3.2 Building Blocks of HNSW	18
3.3.3 Description of HNSW	21
3.3.4 Analysis of HNSW	26
4 Vector Databases in Practice	29
4.1 Systems and Tools	29
4.1.1 FAISS	29

4.1.2	Milvus	30
4.1.3	Weaviate	30
4.2	Practical Applications	30
4.2.1	Image Search	31
4.2.2	Recommendation Systems	34
4.2.3	Document Retrieval and Analysis	34
Appendices		36
A Code Listings		37
A.1	Dataset and Embeddings	37
A.2	Variables Extraction	38
A.3	Milvus Connection	39
A.4	Weaviate Connection	39
Bibliography		40

List of Figures

2.1	Illustration of the embedding process (a) and semantic similarity (b).	4
3.1	LSH hashing process. Each data point $p \in P$ is projected via hash functions g_j into buckets across l hash tables.	9
3.2	Initial set of points in 2D space.	12
3.3	Initial and recursive splits in the construction of an Annoy tree.	13
3.4	Final partitions of the space with a maximum of $K = 10$ points per leaf.	13
3.5	Binary tree corresponding to the final partitioning of the space.	14
3.6	Query point within the partitioned space.	14
3.7	Traversal path in the binary tree for the query point.	15
3.8	Two trees in the Annoy forest. Each tree is generated independently with a random set of splits.	16
3.9	Candidate points and distance evaluation during Annoy forest search.	17
3.10	Graph representation of an NSW structure. Filled circles represent data points, while the entry point and the query point are indicated as an empty circle and a target circle, respectively. Thin edges represent short-range connections preserving local proximity, and thick edges represent long-range connections ensuring overall graph connectivity and enabling polylogarithmic search complexity. Arrows illustrate the path of the greedy search from the entry point to the query.	19
3.11	Voronoi diagram illustration: (a) data points, (b) nearest-neighbor partitioning of the plane.	20
3.12	Delaunay triangulation constructed on top of the Voronoi diagram, connecting points whose Voronoi regions share a boundary.	21
3.13	Graph representation of an HNSW structure. Filled circles represent data points, while the entry point and the query point are indicated as an empty circle and a target circle, respectively. Arrows illustrate the path of the greedy search across layers, starting from the upper layer and descending to lower layers. Upper layers contain longer links and fewer elements, while lower layers contain shorter links and more elements.	22

List of Tables

2.1	Common similarity measures in vector databases.	5
-----	---	---

List of Listings

4.1	<i>k</i> -NN search using FAISS HNSW approach.	31
4.2	<i>k</i> -NN search using Milvus HNSW approach.	32
4.3	<i>k</i> -NN search using Weaviate HNSW approach.	33
A.1	Dataset preparation and embeddings extraction.	37
A.2	Extraction of variables used in demonstration examples.	38
A.3	Connection to Milvus cluster from Google Colab.	39
A.4	Connection to Weaviate cluster from Google Colab.	39

Chapter 1

Introduction

1.1 Background

The study of vector databases and approximate nearest neighbor search has gained significant attention in recent years due to the rapid growth of unstructured, high-dimensional data in domains such as machine learning, computer vision, and natural language processing. Traditional relational databases and exact search methods often fail to provide efficient retrieval mechanisms in these contexts. This is primarily due to the *curse of dimensionality*, which makes high-dimensional nearest neighbor search computationally expensive and impractical [1].

Vector representations, referred to as embeddings, have become a central concept in enabling the effective processing and analysis of unstructured data. By mapping complex objects, such as images, text, or audio, into fixed-length numerical vectors, embeddings allow the application of geometric and algebraic techniques for similarity measurement and data retrieval. The proliferation of embeddings has, in turn, spurred the development of specialized algorithms and data structures to facilitate efficient similarity search, including hash-based, tree-based, and graph-based methods [2].

Vector databases, as platforms designed to store, index, and query embeddings at scale, represent a practical response to these challenges. They integrate algorithmic solutions with software infrastructure to provide high-performance retrieval while maintaining acceptable accuracy levels. Prominent systems, such as FAISS, Milvus, and Weaviate, exemplify the current state-of-the-art in vector data management, offering a range of functionalities from standalone libraries to fully managed database systems [3, 4, 5].

Given the increasing importance of similarity search across research and industry applications, including recommendation systems, image and video retrieval, chatbots, and document analysis, the exploration of both the theoretical and practical aspects of vector databases is timely and highly relevant. This thesis therefore investigates the foundational principles, representative algorithms, and practical tools that enable effective handling of high-dimensional vector data.

1.2 Problem Statement

The increasing volume of unstructured data in modern information systems poses a significant challenge for efficient retrieval and analysis. Traditional relational databases and exact search methods struggle to scale when handling high-dimensional data representations, such as embeddings used in machine learning, computer vision, and natural language processing. As the dimensionality of the data grows, exact nearest neighbor search becomes computationally expensive and impractical.

Approximate nearest neighbor methods and vector database systems have emerged as practical solutions to address these challenges, offering efficient retrieval while maintaining acceptable levels of accuracy. Despite their growing adoption, a comprehensive understanding of the algorithms underlying these systems, as well as the practical considerations for their usage, is still required. This thesis addresses this gap by investigating both the theoretical foundations and the practical applications of vector databases, focusing on the principles, algorithms, and tools that enable efficient similarity search in high-dimensional spaces.

1.3 Research Objectives

The primary objective of this thesis is to investigate the theoretical foundations and practical applications of vector databases, focusing specifically on similarity search in high-dimensional vector spaces. The research aims to:

- Provide an overview of vector data representation and the underlying principles for efficient manipulation and querying of unstructured data.
- Examine the fundamental algorithms for approximate nearest neighbor search and analyze their characteristics, advantages, and limitations.
- Explore prominent tools and systems for managing vector data, highlighting their functionalities and demonstrating their practical usage in real-world scenarios.
- Illustrate the application of vector databases through a practical example, thereby bridging the gap between theoretical concepts and their implementation.

1.4 Methodology Overview

The methodology of this thesis began with a preliminary investigation of vector databases to gain a general understanding of their significance, prevalence, and practical applications. HNSW, LSH, and Annoy were identified as widely cited and representative of different classes of approximate nearest neighbor methods. These algorithms were studied in depth through scientific literature as well as their official open-source implementations, to clarify their operational characteristics.

Insights gained from this investigation informed the development of the theoretical background, providing a structured overview of vector data representation and nearest neighbor search concepts.

The practical part of the study involved examining popular vector management systems and tools, namely FAISS, Milvus, and Weaviate. Embeddings were generated from a selected dataset to demonstrate the use of these systems for similarity search tasks in a controlled, illustrative setting.

1.5 Structure of the Thesis

The thesis is organized into four main chapters:

1. *Introduction*
2. *Theoretical Background*
3. *Selected Algorithms*
4. *Vector Databases in Practice*

Introduction The first chapter outlines the background, problem statement, research objectives, methodology overview, and the structure of the thesis. These sections provide the necessary context and motivation for the work.

Theoretical Background The second chapter introduces the concept of unstructured data and highlights the importance of appropriate representations in facilitating their analysis and manipulation. It then defines the nearest neighbor (NN) search problem, beginning with the exact NN formulation and the challenges it poses in high-dimensional spaces, which motivates the introduction of approximate NN methods and the discussion of the efficiency-accuracy trade-offs they offer. A foundation and motivation are thus established for studying approximate NN algorithms, some of which are examined in the following chapter.

Selected Algorithms The third chapter examines selected approximate nearest neighbor algorithms in detail. It discusses LSH as a representative hash-based method, Annoy as a tree-based method, and HNSW as a graph-based method, thereby providing insight into each of the approaches. This provides an understanding of the inner workings of vector databases, which are further explored in the following chapter.

Vector Databases in Practice The fourth chapter presents the most prominent systems (Milvus, Weaviate) and tools (FAISS) for working with embeddings, highlighting practical use cases and illustrating their application through a demonstration example. This effectively completes the thesis by linking theoretical concepts with practical implementation.

Chapter 2

Theoretical Background

2.1 Unstructured Data

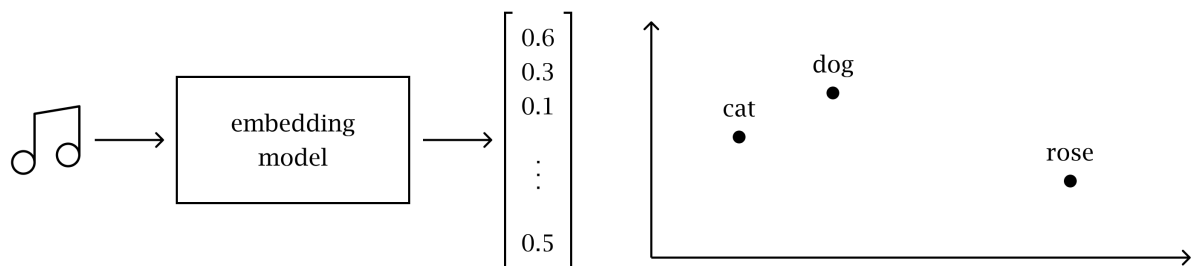
Unstructured data refers to information that does not follow a predefined schema or tabular format, making it inherently more complex to process and analyze than structured (typically stored in SQL databases) or semi-structured (typically managed by NoSQL databases) data. Examples include text documents, images, audio and video files, sensor logs, scientific data, and code, which vary widely in format, size, and origin and often require specialized techniques to extract meaningful insights.

2.1.1 Embeddings

Embeddings are numerical representations of data mapped into a continuous vector space (Figure 2.1a), where semantically similar objects are placed close together (Figure 2.1b). They bridge the gap between discrete, symbolic data and differentiable machine learning models, representing complex objects as fixed-length vectors. High-quality embeddings capture relevant features of the original data, supporting tasks such as classification, clustering, and similarity search [2].

2.1.2 Distance Metrics

Choosing an appropriate distance metric (Table 2.1) ensures that embeddings accurately reflect the semantic similarity of the original unstructured data.



(a) Unstructured data processed through a model to generate embeddings.

(b) Semantically similar objects appear close together, while dissimilar ones are farther apart.

Figure 2.1: Illustration of the embedding process (a) and semantic similarity (b).

Table 2.1: Common similarity measures in vector databases.

Metric	Formula
l_p (Minkowski)	$\ x - y\ _p = (\sum_{i=1}^d x_i - y_i ^p)^{1/p}$
l_1 (Manhattan)	$\ x - y\ _1 = \sum_{i=1}^d x_i - y_i $
l_2 (Euclidean)	$\ x - y\ _2 = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$
l_∞ (Chebyshev)	$\ x - y\ _\infty = \max_i x_i - y_i $
Cosine distance	$d_{cos}(x, y) = 1 - (x \cdot y) / (\ x\ _2 \cdot \ y\ _2)$
Dot (inner) product	$s_{dot}(x, y) = \sum_{i=1}^d x_i \cdot y_i$
Hamming distance	$d_H(x, y) = \sum_{i=1}^d 1(x_i \neq y_i)$

2.2 Nearest Neighbor Search

2.2.1 Exact Nearest Neighbor Search

Definition Given a set P of points in a normed space l_p^d , preprocess P so as to efficiently answer queries by returning a point $p \in P$ closest to a query point q .

This definition generalizes naturally to finding $k > 1$ nearest neighbors. In the k -NN search problem, k points $p_1, \dots, p_k$ are returned such that each p_i is the i -th closest point to q [6].

Brute Force Approach The most straightforward approach for exact nearest neighbor search is the brute force (BF), or exhaustive search approach, which computes the distance between a query point and all points in the dataset to return the closest one. However, the computational cost of a single BF k -NN query using a chosen l_p^d norm is $O(n \cdot d)$ [1].

Structured Indexing Methods Several structured indexing methods have been proposed for exact k -NN search to efficiently handle low-dimensional datasets [1]:

- **R-tree** and its variants (R*-tree, Hilbert R-tree, PR-tree): A height-balanced tree in which leaf nodes contain entries with an n -dimensional rectangle representing the bounding box of the associated spatial object, and non-leaf nodes contain entries that cover all rectangles in the entries of the lower node [7].
- **KD-tree**: A binary space-partitioning tree where each node stores a point from the dataset, and non-leaf nodes implicitly define a hyperplane that partitions the space into two child subtrees. At each level, the dimension with the largest spread is chosen, and the dataset is split at the median along that dimension [8].

- **Ball-tree:** A metric space-partitioning tree where each node defines a hypersphere containing a subset of the dataset. Each internal node is split in two steps: (1) the farthest point from the centroid of the points is selected as the first child centroid, and (2) the farthest point from this first centroid is selected as the second child centroid [9].
- **VP-tree** (and MVP-tree): A metric space-partitioning tree where each node selects a vantage point from its subset of points and partitions the points based on the median distance to this vantage point. Points closer than the median go to one child subtree, and points farther go to the other. This process is applied recursively, with distinct vantage points at each node, until leaf nodes containing a small number of points are reached [10].

Curse of Dimensionality As discussed above, these k -NN methods perform well in low-dimensional settings, but their efficiency degrades as the number of dimensions increases [1].

The term *curse of dimensionality* refers to the exponential growth of the volume of a space as the number of dimensions increases. This growth has significant implications for nearest neighbor search: as the dimensionality rises, data points become increasingly sparse. It causes distances between points to become increasingly similar, a phenomenon known as the *concentration of measure*. As a result, the difference between the closest and farthest neighbors diminishes, reducing the effectiveness of standard distance metrics such as the l_p^d norm.

Moreover, the impact of dimensionality depends on the relevance of the features: adding meaningful dimensions can improve contrast and preserve useful rankings of neighbors, while irrelevant dimensions reduce contrast and make distance-based queries less reliable [11].

2.2.2 Approximate Nearest Neighbor Search

Definition Given a set P of points in a normed space l_p^d , preprocess P so as to efficiently answer queries by returning a point $p \in P$ for any given query point q , such that

$$d(p, q) \leq (1 + \varepsilon) \cdot d(q, P),$$

where $d(q, P)$ is the distance from q to its closest point in P .

This definition generalizes naturally to finding $k > 1$ approximate nearest neighbors. In the approximate k -NN search problem, k points $p_1, \dots, p_k$ are returned such that the distance of p_i to the query q is at most $(1 + \varepsilon)$ times the distance from the i -th nearest point to q [6].

Motivation for Approximation In practical applications, the choice of features and distance metric is often heuristic, aiming to formalize what is fundamentally an intuitive notion of similarity. Consequently, insisting on exact nearest neighbors can be unnecessarily strict. Instead, determining an ε -approximate nearest neighbor for a reasonable constant ε is usually sufficient for most purposes [12].

Approximate nearest neighbor search aims to reduce the computational cost of similarity queries by relaxing the strict correctness requirement: the returned objects may not be the exact nearest neighbors. In practice, users often prefer faster responses that are sufficiently accurate rather than exact results, particularly in high-dimensional or large-scale datasets. The success of approximate methods relies on effectively balancing the trade-off between query efficiency and result quality [13].

Classification of Approximate Techniques Approximate nearest neighbor methods can be classified along several dimensions that reflect their applicability and design choices [13]:

- **Type of space** - Techniques can be specialized for different kinds of data spaces:
 - Vector spaces with a specific l_p^d distance.
 - Vector spaces without restriction on the distance function.
 - Metric spaces, where only the metric properties are assumed.
- **Approximation mechanism** - How efficiency is achieved:
 - Changing space: The search space is transformed or reduced (e.g., dimensionality reduction, bit approximations).
 - Reducing comparisons: The search examines fewer candidates, either by
 - * aggressive pruning, or
 - * early stopping.
- **Quality guarantees** - Whether the method provides bounds on approximation errors:
 - No guarantees: Heuristic approaches without formal bounds.
 - Deterministic guarantees: Methods with provable maximum error bounds.
 - Probabilistic guarantees: Techniques that guarantee correctness with a certain probability, either
 - * parametric, or
 - * non-parametric.
- **User interaction** - How parameters can be specified at query time:
 - Static approach: Parameters are fixed at index construction; changing them requires rebuilding.
 - Interactive approach: Users can adjust parameters dynamically at query time to trade off accuracy and efficiency.

* * *

Unstructured data requires specialized representations to enable meaningful analysis. Embeddings map complex objects into a continuous vector space where semantic similarity can be quantified, while distance metrics determine how similarity is measured. Nearest neighbor search provides a framework for retrieving similar items, with exact methods offering precision and approximate methods trading some accuracy for efficiency. These concepts form the foundation for exploring specific algorithms for approximate nearest neighbor search, exemplified by hash-based, tree-based, and graph-based approaches.

Chapter 3

Selected Algorithms

3.1 Locality-Sensitive Hashing (LSH)

3.1.1 Introduction to LSH

Locality-Sensitive Hashing (LSH) is a hashing-based approach for approximate nearest neighbor (ANN) search. The concept of LSH was first introduced by Indyk and Motwani in 1998 [12] and later improved and generalized by Gionis, Indyk, and Motwani in 1999 [6].

The key idea of LSH is to design hash functions such that similar objects are mapped to the same bucket with high probability, while dissimilar objects are unlikely to collide. This property allows sublinear query time, as the search can be restricted to the elements stored in the buckets corresponding to the query point [6].

LSH significantly improved running time over previous methods for searching in high-dimensional spaces based on hierarchical tree decompositions [6]. Although more recent graph-based and quantization-based methods, such as HNSW or IVF-PQ, often outperform LSH in practice, it remains a foundational algorithm with strong theoretical guarantees and continues to influence modern research in approximate nearest neighbor search.

3.1.2 Description of LSH

To simplify the presentation, the algorithm is described [6] under two assumptions: (i) distances are measured with the l_1 norm, and (ii) all coordinates of the data points are positive integers. After these transformations, the minimum interpoint distance is guaranteed to be one.

Let C be the largest coordinate value in all points in P . Each point $p = (x_1, \dots, x_d)$ is conceptually mapped to a binary vector in dimension $d' = C \cdot d$ by concatenating the unary representations of its coordinates:

$$v(p) = \text{Unary}_C(x_1) \parallel \dots \parallel \text{Unary}_C(x_d),$$

where $\text{Unary}_C(x)$ is a binary string of x ones followed by $C - x$ zeros. This mapping preserves distances: the l_1 distance in the original space equals the d_H distance in the Hamming space $H^{d'}$. In practice, the unary expansion is not materialized explicitly; it is used only as a conceptual framework for defining the algorithms.

Hash Functions The LSH scheme uses l different hash tables, each associated with its own hash function g_j , for $j = 1, \dots, l$. For each hash table, the function g_j is defined by selecting k coordinates uniformly at random (with replacement) from $\{1, \dots, d'\}$ and projecting the binary vector $v(p)$ onto these positions, producing $g_j(p)$. The parameter k is chosen to maximize the probability that a point close to a query q falls into the same bucket as q , while minimizing the probability that a distant point collides in the same bucket. Each point $p \in P$ is then stored in the corresponding bucket $g_j(p)$ across all l tables (Figure 3.1).

Since the number of buckets can be very large, a second level of hashing is used to compress them into a hash table of fixed size M , with each bucket limited to a maximum capacity of B [6]. The selection of the parameters l and k is discussed in the next section.

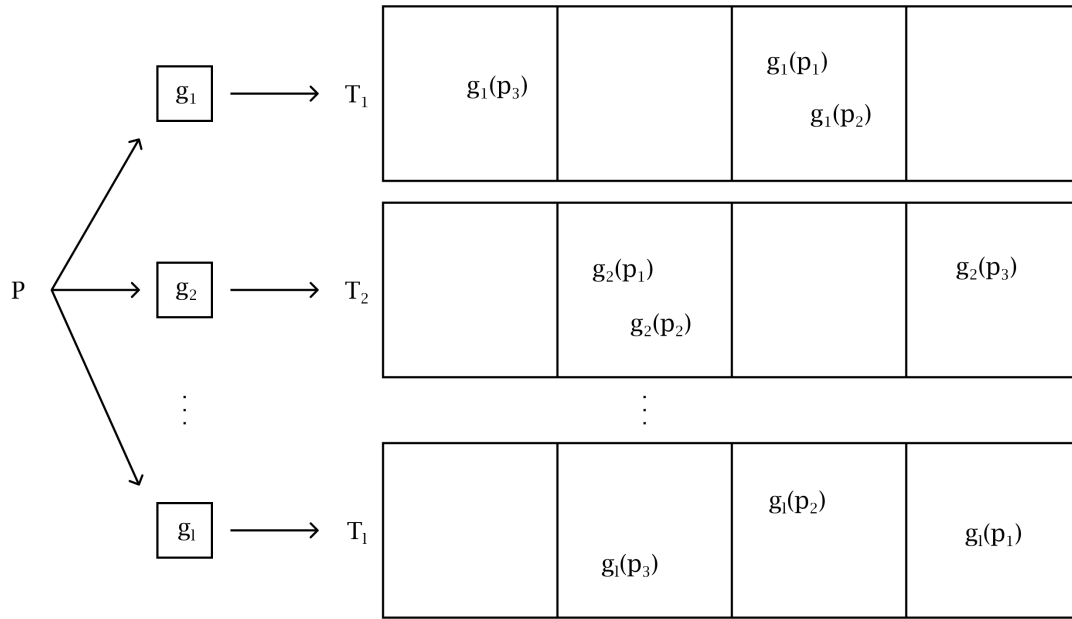


Figure 3.1: LSH hashing process. Each data point $p \in P$ is projected via hash functions g_j into buckets across l hash tables.

Indexing

The indexing stage (Algorithm 1) constructs the l hash tables and distributes the dataset points into the appropriate buckets:

Algorithm 1 BUILD-INDEX [6]

Require: points P , number of hash tables l

Ensure: hash tables T

- 1: **for** $i \leftarrow 1 \dots l$ **do**
 - 2: generate random hash function $g_i(\cdot)$ and initialize hash table T_i
 - 3: **for each** $p \in P$ **do**
 - 4: store point p in bucket $g_i(p)$ of hash table T_i
 - 5: **end for**
 - 6: **end for**
-

Querying

To answer a query, the algorithm (Algorithm 2) hashes the query point q into each of the l tables, retrieves candidate points from the corresponding buckets, and returns the k nearest neighbors from the candidate set:

Algorithm 2 K-NN-SEARCH [6]

Require: l hash tables T , query point q , number of nearest neighbors k

Ensure: up to k approximate nearest neighbors of q

```

1:  $S \leftarrow \emptyset$  ▷ candidate set
2: for  $i \leftarrow 1 \dots l$  do
3:    $S \leftarrow S \cup \{\text{points in bucket } g_i(q) \text{ of hash table } T_i\}$ 
4: end for
5: return up to  $k$  nearest neighbors of  $q$  in  $S$ 

```

3.1.3 Analysis of LSH

The main intuition behind LSH is that the probability of two points colliding in the same hash bucket decreases as the distance between them increases, which is formalized as follows [6]. Let $D(\cdot, \cdot)$ be a distance function on a set S , and for any $p \in S$ let $\mathcal{B}(p, r)$ denote the set of elements from S within distance r from p . A family $\mathcal{H} = \{h : S \rightarrow U\}$ is called (r_1, r_2, p_1, p_2) -sensitive for $D(\cdot, \cdot)$ if for any $p, q \in S$:

- If $p \in \mathcal{B}(q, r_1)$, then $\Pr_{\mathcal{H}}[h(q) = h(p)] \geq p_1$.
- If $p \notin \mathcal{B}(q, r_2)$, then $\Pr_{\mathcal{H}}[h(q) = h(p)] \leq p_2$.

For a locality-sensitive family to be useful, it must hold that $p_1 > p_2$ and $r_1 < r_2$. This ensures that points close to the query are likely to collide in a hash bucket, while distant points are unlikely to.

Hamming Locality-Sensitive Family When the distance function is the Hamming distance, the family of projections on one coordinate is locality-sensitive. Specifically, let $S = H^{d'}$ and $D(p, q) = d_H(p, q)$ for $p, q \in H^{d'}$. Then for any $r, \epsilon > 0$, the family

$$\mathcal{H}_{d'} = \left\{ h_i : h_i((b_1, \dots, b_{d'})) = b_i, i = 1, \dots, d' \right\}$$

is

$$\left(r, r(1 + \epsilon), 1 - \frac{r}{d'}, 1 - \frac{r(1 + \epsilon)}{d'} \right)\text{-sensitive.}$$

That is, points within distance r collide with probability at least $1 - r/d'$, while points farther than $r(1 + \epsilon)$ collide with probability at most $1 - r(1 + \epsilon)/d'$ [6].

Arbitrary Locality-Sensitive Family The algorithm can be generalized to any locality-sensitive family $\mathcal{H}$. In this case, each hash function is defined as

$$g_i(p) = (h_{i_1}(p), \dots, h_{i_k}(p)),$$

where $h_{i_1}, \dots, h_{i_k}$ are randomly chosen from $\mathcal{H}$ with replacement. As before, l such functions $g_1, \dots, g_l$ are constructed to build l hash tables [6].

Correctness via the (r, ε) -Neighbor Problem Formally, the algorithm is shown [6] to correctly solve the so-called (r, ε) -Neighbor problem, which asks whether there exists a point p within a fixed distance $r_1 = r$ of the query q , or whether all points are at least distance $r_2 = (1 + \varepsilon)r$ away. In particular, the LSH algorithm solves this problem for a proper choice of parameters k and l , depending on r and ε . More precisely, for the set of points $P' \subseteq P$ farther than r_2 from q , the algorithm correctly solves the (r, ε) -Neighbor problem if the following two properties hold:

- P1** If there exists a point within distance r_1 of the query, then at least one of the hash functions $g_1, \dots, g_l$ maps it to the same bucket as the query.
- P2** The total number of hash buckets pointed to by the query that contain only points from P' is bounded by $c \cdot l$, for some constant c .

Intuitively, P1 ensures that close points are likely to be retrieved by the query, while P2 guarantees that the number of far points incorrectly considered is limited, keeping the search efficient. As proven in [6], by choosing the parameters:

$$k = \log_{\frac{1}{p_2}} \frac{n}{B}, \quad l = \left(\frac{n}{B}\right)^\rho, \quad \rho = \frac{\ln \frac{1}{p_1}}{\ln \frac{1}{p_2}}$$

both properties P1 and P2 hold with probability at least $1/2 - 1/e \geq 0.132$, ensuring the correctness of the LSH algorithm.

When instantiated for the Hamming locality-sensitive family, it can be shown that

$$\rho \leq \frac{1}{1 + \varepsilon}$$

under the assumption that $r < d'/\ln n$ which can be easily satisfied. Thus, the expected query time of the LSH algorithm is sublinear in the number of points n . Specifically, for a dataset of n points in dimension d , the query time is $O(d \cdot n^{1/(1+\varepsilon)})$ [6].

Connection to the ε -NNS Problem While the (r, ε) -Neighbor problem is central for the theoretical analysis of LSH, in practice the main objective is to solve the ε -Nearest Neighbor Search (ε -NNS) problem. This problem can be reduced to a sequence of (r, ε) -Neighbor instances for different values of r , specifically $r_0, r_0(1 + \varepsilon), r_0(1 + \varepsilon)^2, \dots, r_{\max}$, where r_0 and $r_{\max}$ denote the smallest and the largest possible distance between the query and the dataset, respectively [6]. By testing successive (r, ε) -Neighbor instances for increasing values of r , one can identify the smallest radius at which a neighbor is detected.

However, empirical evidence suggests that in most cases a single fixed value of r is sufficient, since the distribution of distances between queries and the dataset typically depends more on the intrinsic properties of the data than on the particular query point. Consequently, in practice a fixed choice of r is often adopted, which also fixes the values of k and l for the entire dataset [6].

3.2 Approximate Nearest Neighbors Oh Yeah (Annoy)

3.2.1 Introduction to Annoy

Approximate Nearest Neighbors Oh Yeah (Annoy) is a tree-based approach for approximate nearest neighbor (ANN) search. It was developed by Bernhardsson during Spotify's Hack Week in 2013 [14] for music recommendation.

An Annoy tree is constructed using random projections: at each intermediate node, a hyperplane is defined by sampling two points from the subset and placing it equidistant from them, thereby dividing the space into two subspaces. The recursive process continues until a single tree is formed [14].

Although Annoy does not provide formal theoretical guarantees like LSH or HNSW, it has demonstrated good empirical performance. By offering a favorable trade-off between query speed, memory usage, and accuracy [15], it is often used in practice and is also referenced in research on approximate nearest neighbor search.

The Annoy implementation is available at GitHub¹.

3.2.2 Description of Annoy

The description provided in this section is based on the official Spotify Annoy repository [14] and Bernhardsson's presentation on Annoy [16].

Indexing: Tree

To illustrate the construction of an Annoy index, a set of points representing vectors in the space is shown (Figure 3.2). For visualization purposes, a 2-dimensional example is used; in practice, vectors typically reside in much higher-dimensional spaces.

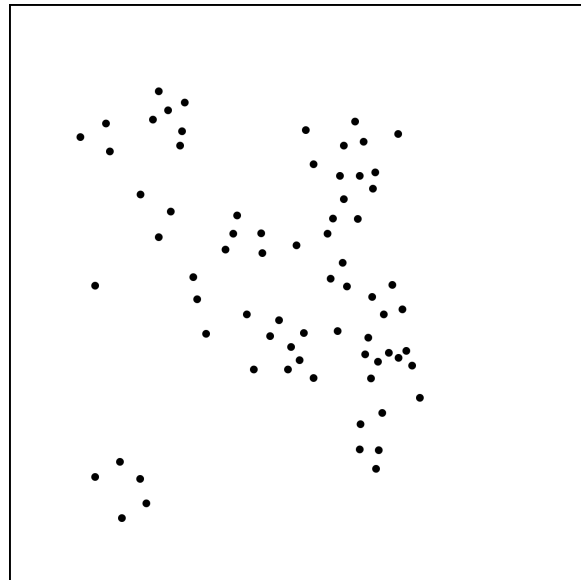


Figure 3.2: Initial set of points in 2D space.

¹<https://github.com/spotify/annoy>

The goal is to construct a data structure that allows nearest neighbor queries to be answered in sublinear time. To support this, Annoy builds a binary tree in which each node represents a random partition of the space. The process begins with a single split of the space (Figure 3.3a).

Each split is performed by randomly selecting two points and constructing a hyperplane equidistant from them. This hyperplane divides the current subspace into two halves, with each half assigned to a child node in the binary tree. The partitioning continues recursively for each resulting subspace (Figure 3.3b). In practice, the hyperplane itself does not need to be explicitly computed; each vector is assigned to a side based on which of the two selected points it is closer to.

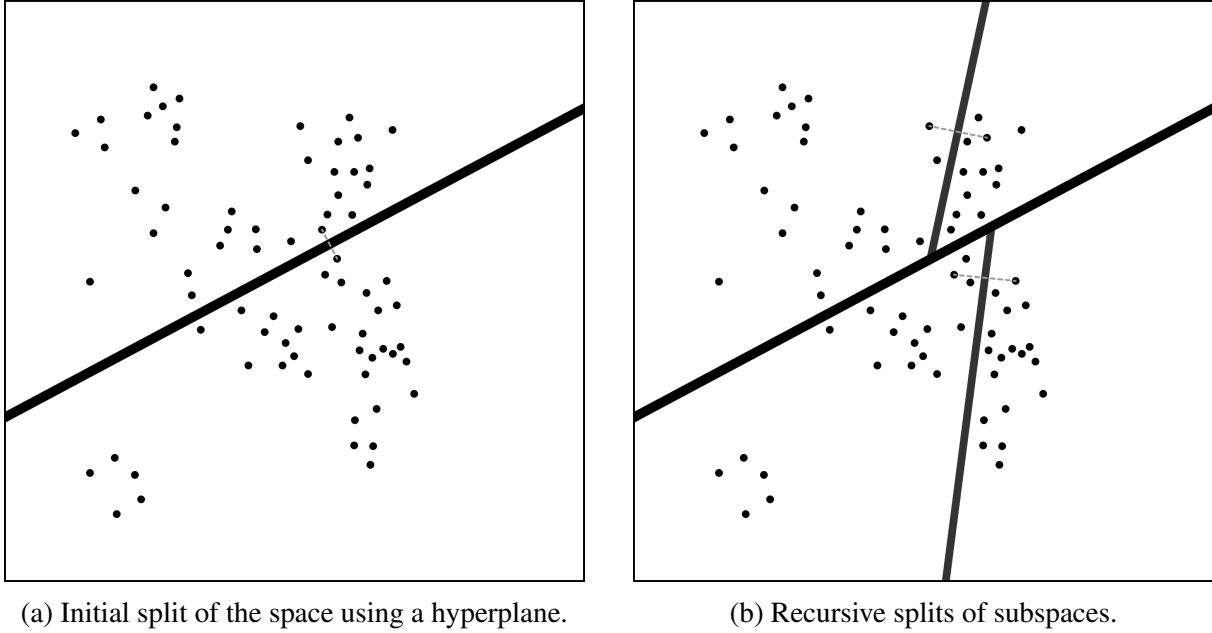


Figure 3.3: Initial and recursive splits in the construction of an Annoy tree.

The recursive partitioning proceeds until each leaf node contains at most K points (Figure 3.4).

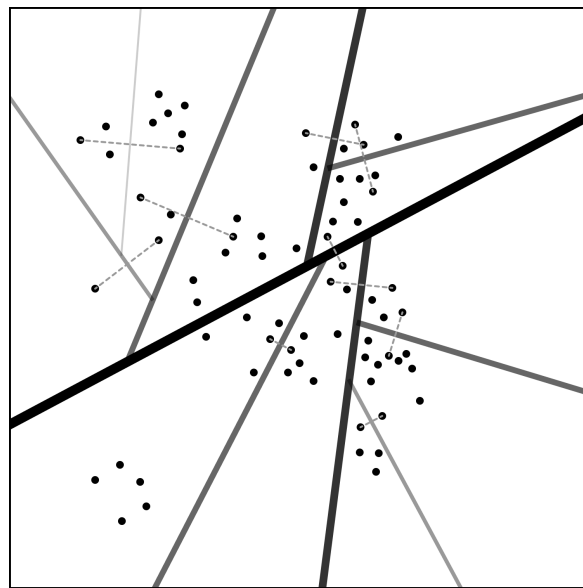


Figure 3.4: Final partitions of the space with a maximum of $K = 10$ points per leaf.

The structure of the binary tree (Figure 3.5) reflects the hierarchical partitioning of the space. Each leaf node in the tree corresponds to one of the final subspaces.

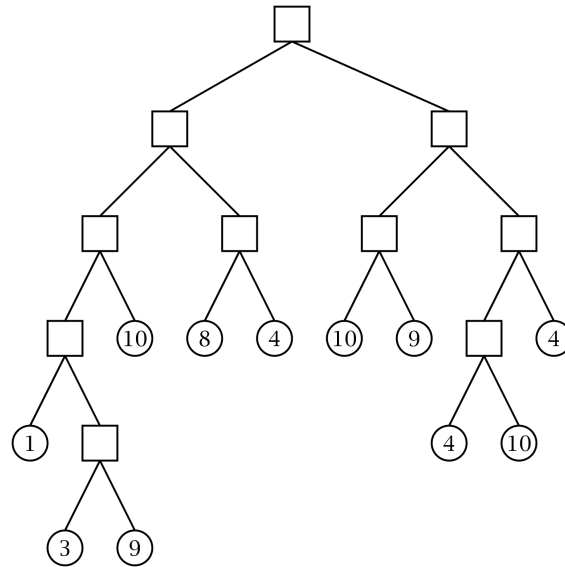


Figure 3.5: Binary tree corresponding to the final partitioning of the space.

Points that are close to each other are likely to remain within the same or neighboring leaf nodes. Consequently, hyperplanes rarely separate points that are near each other, preserving local proximity within the tree structure.

Querying

Once the space has been partitioned, a query point is evaluated within the structure to search for its nearest neighbors (Figure 3.6).

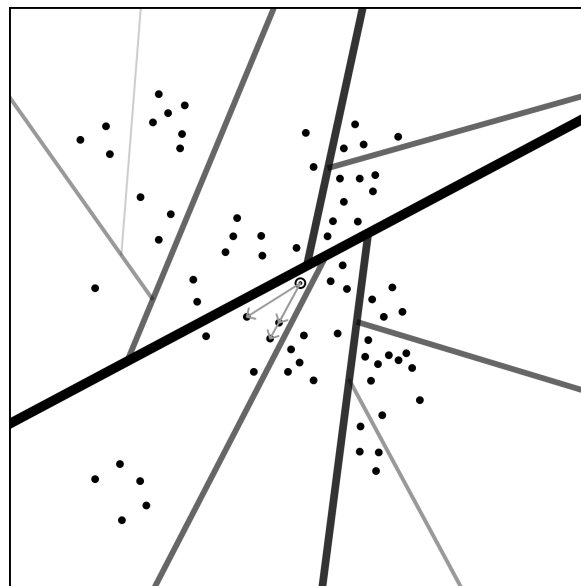
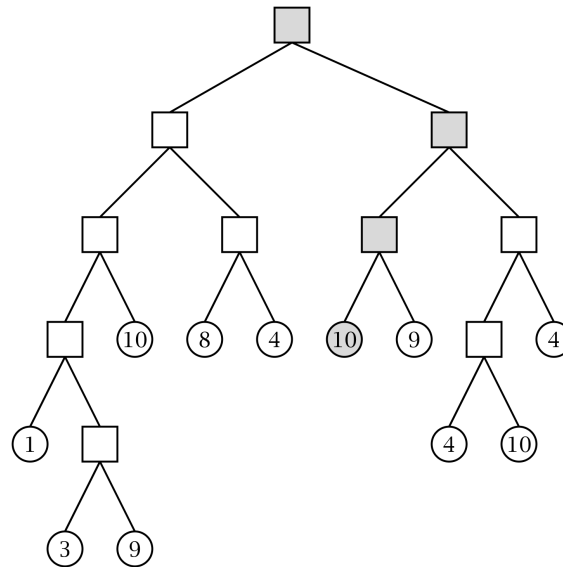
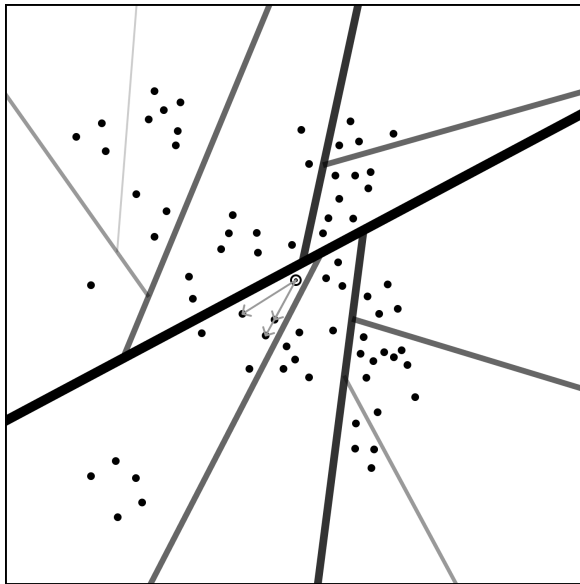
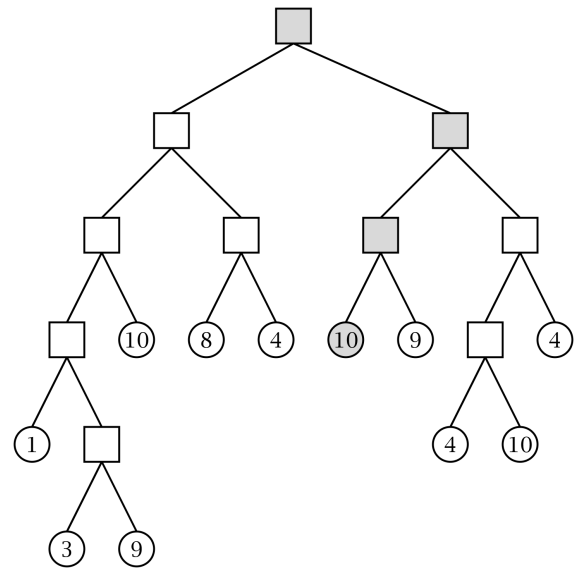


Figure 3.6: Query point within the partitioned space.

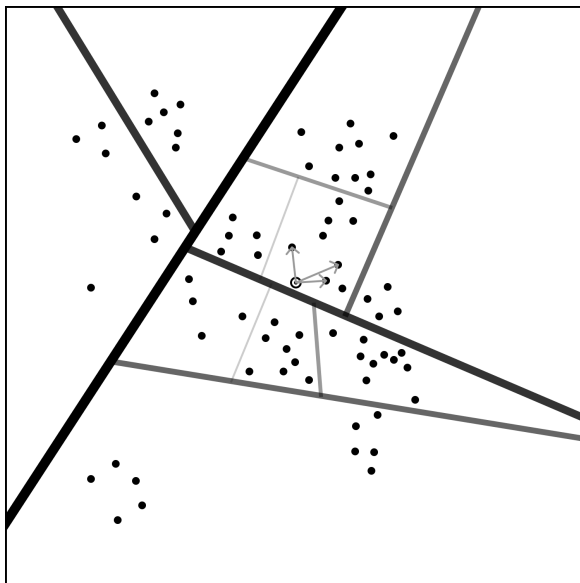




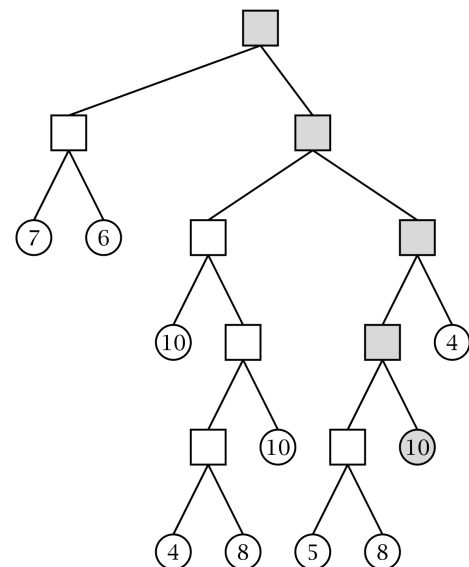
(a) Final partitioning of the first tree.



(b) Binary tree corresponding to the first partitioning.



(c) Final partitioning of the second tree.



(d) Binary tree corresponding to the second partitioning.

Figure 3.8: Two trees in the Annoy forest. Each tree is generated independently with a random set of splits.

Since each tree contains all points, some points may appear in multiple leaf nodes across different trees. The union of these leaf nodes identifies candidate points that are likely to include the nearest neighbors (Figure 3.9a).

Distances from the query point to these candidate points are computed during the simultaneous traversal of all trees, with a single priority queue prioritizing nodes and facilitating retrieval of the top n nearest neighbors (Figure 3.9b).

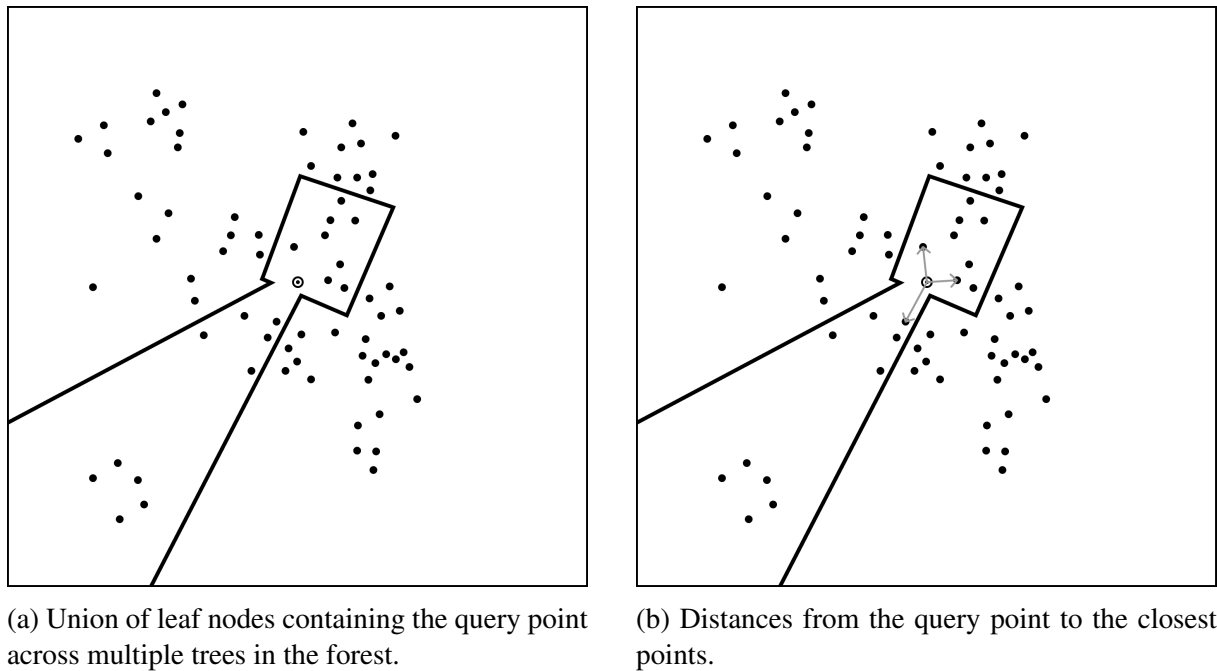


Figure 3.9: Candidate points and distance evaluation during Annoy forest search.

Increasing the number of trees generally improves recall by providing more opportunities to encounter favorable splits. The practical upper limit is typically constrained by available memory resources.

3.2.3 Analysis of Annoy

Parameters

There are two main parameters that control the performance of Annoy: the number of trees n_trees and the number of nodes inspected during query execution $search_k$ [14].

n_trees Parameter The parameter n_trees defines the number of trees in the Annoy forest and is set at build time. Increasing n_trees generally improves recall, since more independent random partitions raise the chance of obtaining favorable splits. This improvement, however, comes at the cost of longer index construction and increased memory usage. In practice, the value of n_trees is typically chosen as large as possible within available memory limits [14].

$search_k$ Parameter The parameter $search_k$ specifies the number of nodes inspected during query execution and is set at runtime. Larger values improve recall by considering more candidate points, but also increase query latency. If not explicitly specified, $search_k$ defaults to $n \times n_trees$, with n denoting the number of requested nearest neighbors. In practice, $search_k$ is usually set as large as possible given the time constraints for query processing [14].

Multiple Processes and Memory Sharing

A distinctive feature of Annoy is its ability to share a single index across multiple processes. This is achieved by creating static file-based indexes that can be memory-mapped (mmap) by any number of processes. By decoupling index creation from loading, the same index file can be distributed or passed around, allowing immediate lookups without rebuilding the structure.

This approach is particularly useful in high-performance or distributed environments, where multiple CPUs or workers need to access the index concurrently. Only a single copy of the index needs to be built, minimizing both construction overhead and memory usage. Moreover, because index creation is separate from lookup, no additional vectors can be added once the index is built [14].

3.3 Hierarchical Navigable Small World (HNSW)

3.3.1 Introduction to HNSW

Hierarchical Navigable Small World (HNSW) is a graph-based approach for approximate nearest neighbor (ANN) search, built on navigable small-world (NSW) networks with a controllable hierarchical structure. It was initially introduced by Malkov and Yashunin as an arXiv preprint in 2016 [17] and later published in the peer-reviewed *IEEE Transactions on Pattern Analysis and Machine Intelligence* in 2020 [18].

HNSW incrementally builds a multi-layered structure composed of hierarchical proximity graphs, where each layer corresponds to a nested subset of the stored elements. The maximum layer in which an element appears is selected randomly according to an exponentially decaying probability distribution [17].

Starting the search from the upper layer, together with utilizing scale separation, boosts performance compared to NSW and allows logarithmic complexity scaling [17]. Performance evaluations have shown that HNSW remains one of the state-of-the-art approaches for vector search [19], leading to its adoption in various vector databases and search systems.

The HNSW implementation is available at GitHub².

3.3.2 Building Blocks of HNSW

Navigable Small World (NSW)

Navigable Small World (NSW), predecessor of HNSW, is a graph-based approach for approximate nearest neighbor (ANN) search. It combines two key ideas: short-range connections that approximate the Delaunay triangulation, and long-range connections that ensure the small-world navigation property (Figure 3.10 [20]). The short-range edges preserve local proximity information required for greedy search, while long-range edges maintain overall graph connectivity and enable polylogarithmic scalability of search complexity. A distinctive feature of the NSW construction is that long-range links arise naturally: as new elements are inserted consecutively and connected to their nearest neighbors, some of the previously added edges gradually become long-range connections [20].

An important limitation of NSW lies in its routing process, which can be divided into two phases: "zoom-out" and "zoom-in" [17]. In the "zoom-out" phase, the search expands outward from a low-degree node, progressively reaching high-degree nodes (hubs), while in the "zoom-in" phase the routing converges towards the target through increasingly local, short-range connections [21]. Starting the search directly from hubs, thereby skipping the "zoom-out" stage, reduces the probability of getting stuck in a distant false local minimum and improves efficiency. Nevertheless, the overall routing in NSW still exhibits only polylogarithmic complexity. This shortcoming motivated the introduction of a hierarchical structure in HNSW to achieve logarithmic search complexity [17].

²<https://github.com/nmslib/hnswlib>

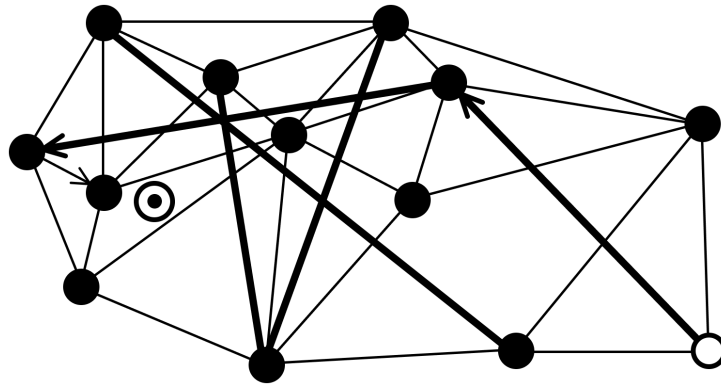


Figure 3.10: Graph representation of an NSW structure. Filled circles represent data points, while the entry point and the query point are indicated as an empty circle and a target circle, respectively. Thin edges represent short-range connections preserving local proximity, and thick edges represent long-range connections ensuring overall graph connectivity and enabling polylogarithmic search complexity. Arrows illustrate the path of the greedy search from the entry point to the query.

The Small World Problem Social psychologist Milgram investigated [22] how individuals are connected into social networks through ties of kinship and acquaintance, and whether short paths exist between any two randomly chosen people. To test this, he devised an experimental method: participants were given the name and basic information of a target person. If they did not personally know the target, they were instructed to forward the folder to a personal acquaintance who was more likely to know the target, restricted to contacts they knew on a first-name basis.

The experiment revealed that the median number of intermediaries required to connect two individuals was approximately 5.5, providing empirical evidence for the small-world phenomenon (popularly known as *six degrees of separation* [23]).

Navigability Computer scientist Kleinberg studied [24, 25] how individuals can efficiently find short paths in large social networks using only local information. He modeled networks following the small-world paradigm, characterized by dense short-range connections and a few random long-range links. While small-world networks guarantee that short paths exist between any two nodes, this does not ensure that a decentralized algorithm can discover them.

Kleinberg showed that there exists a unique value of a parameter controlling the distribution of long-range links at which a greedy algorithm can navigate efficiently, with expected delivery time scaling polylogarithmically with N , where N is the number of nodes. For other values of this parameter, navigation becomes asymptotically inefficient. These results indicate that navigability is not a general property of all small-world networks but only emerges under specific structural conditions.

Proximity Graph Proximity graphs are geometric graphs in which two vertices p and q are connected by an edge (p, q) if and only if a certain exclusion region defined for p and q contains no other points from the vertex set. Depending on the choice of the exclusion region, various types of proximity graphs can be defined, such as the Delaunay triangulation [26].

Voronoi Diagram Given a set of points in the plane (Figure 3.11a [27]), a Voronoi diagram partitions the plane according to the nearest-neighbor rule: each point is associated with the region of the plane that is closest to it (Figure 3.11b [27]). Voronoi diagrams naturally arise in various proximity problems, such as the classical post-office problem, which asks how a fixed set of points can be preprocessed to quickly determine the nearest point to an arbitrary query location, and its generalization to finding the k nearest points [28].

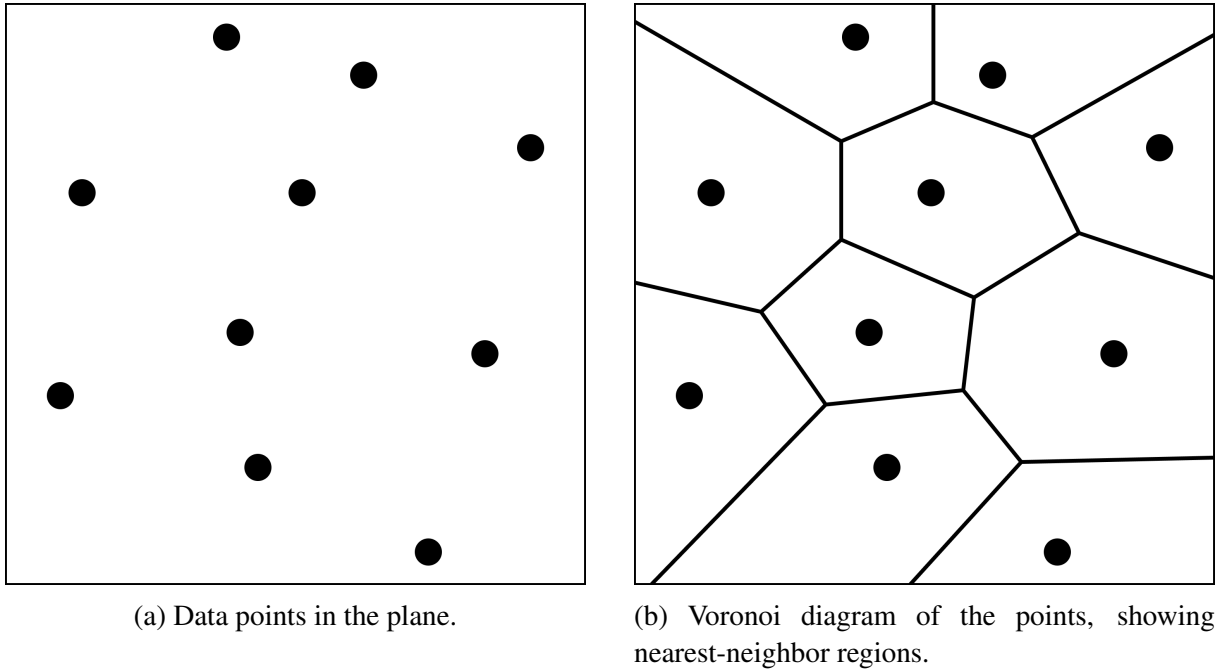


Figure 3.11: Voronoi diagram illustration: (a) data points, (b) nearest-neighbor partitioning of the plane.

Delaunay Triangulation Delaunay triangulation is closely related to Voronoi diagrams: two points are connected by a straight-line edge if and only if their Voronoi regions share a boundary (Figure 3.12 [27]). It can also be defined using the empty-circle property, where triangles are formed such that the circumcircle of each triangle contains no other point, and the set of these edges constitutes the triangulation.

As the geometric dual of the Voronoi diagram, Delaunay triangulation preserves local proximity information by linking neighboring points [28]. A key property of the Delaunay graph is that greedy search is guaranteed to find the true nearest neighbor [17].

Despite this advantage, constructing the Delaunay graph requires prior knowledge of the metric space [29] and suffers from the curse of dimensionality [28]. Moreover, since exactness is not required in approximate nearest neighbor search, maintaining the full Delaunay graph is unnecessary [20].

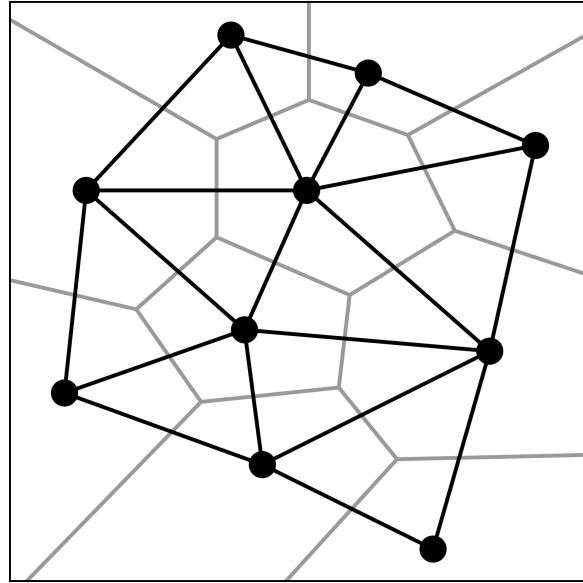


Figure 3.12: Delaunay triangulation constructed on top of the Voronoi diagram, connecting points whose Voronoi regions share a boundary.

Skip List

HNSW can alternatively be interpreted as an extension of the probabilistic skip list structure, where the linked lists are replaced by proximity graphs [17]. This perspective highlights the hierarchical organization of elements across multiple layers, similar to how skip lists allow efficient search through multiple levels of linked nodes.

3.3.3 Description of HNSW

The algorithm organizes elements into multiple layers: upper layers contain longer links and fewer elements, while lower layers contain shorter links and more elements. This design enables efficient search by starting from the upper layer (the "zoom-in" phase), greedily moving toward a local minimum, and then descending to lower layers, restarting from the local minimum found in the previous layer (Figure 3.13 [17]).

Unlike NSW, elements do not need to be shuffled before insertion. Instead, stochasticity is introduced through level randomization, which supports truly incremental indexing even under temporary changes in data distribution [17].

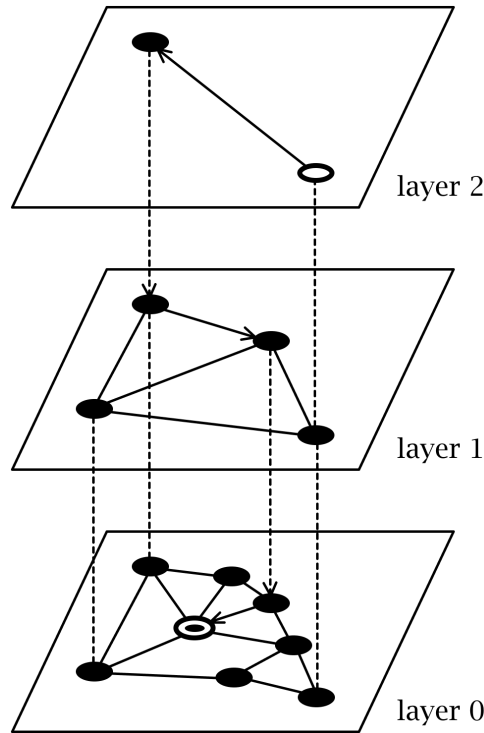


Figure 3.13: Graph representation of an HNSW structure. Filled circles represent data points, while the entry point and the query point are indicated as an empty circle and a target circle, respectively. Arrows illustrate the path of the greedy search across layers, starting from the upper layer and descending to lower layers. Upper layers contain longer links and fewer elements, while lower layers contain shorter links and more elements.

Element Insertion The construction of the HNSW structure is based on consecutive insertions of new elements into the multilayer graph (Algorithm 3). For each element q , a maximum layer l is sampled from a geometric distribution controlled by the parameter m_L .

The insertion begins at the top layer L , where the algorithm performs a greedy search to find the ef closest neighbors of q . The search then proceeds layer by layer down to level $l + 1$, each time using the best candidates from the previous layer as entry points. At layers l down to 0, the algorithm performs a broader greedy search to identify the $ef_{Construction}$ closest neighbors of q , from which up to M are selected to establish bidirectional connections. If the degree of a neighboring element exceeds M_{max} (or M_{max0} at the ground layer), its connections are pruned using the same neighbor selection procedure.

Finally, if q is assigned a level higher than the current maximum, it becomes the new entry point of the structure [17].

Algorithm 3 INSERT [17]

Require: multilayer graph h_{nsw} , new element q , number of established connections M , maximum number of connections per element per layer M_{max} , size of dynamic candidate list $efConstruction$, normalization factor for level generation m_L

Ensure: updated h_{nsw} with element q inserted

```

1:  $W \leftarrow \emptyset$  ▷ list of currently found nearest elements
2:  $ep \leftarrow$  get entry point of  $h_{\text{nsw}}$ 
3:  $L \leftarrow$  level of  $ep$  ▷ top layer in  $h_{\text{nsw}}$ 
4:  $l \leftarrow \lfloor -\ln(\text{unif}(0..1)) \cdot m_L \rfloor$  ▷ level of new element
5: for  $l_c \leftarrow L \dots l + 1$  do
6:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef = 1, l_c)$ 
7:    $ep \leftarrow$  get nearest element from  $W$  to  $q$ 
8: end for
9: for  $l_c \leftarrow \min(L, l) \dots 0$  do
10:   $W \leftarrow \text{SEARCH-LAYER}(q, ep, efConstruction, l_c)$ 
11:   $neighbors \leftarrow \text{SELECT-NEIGHBORS}(q, W, M, l_c)$  ▷ Algorithm 5 or 6
12:  add bidirectional connections from  $neighbors$  to  $q$  at layer  $l_c$ 
13:  for each  $e \in neighbors$  do ▷ shrink connections if necessary
14:     $eConn \leftarrow neighborhood(e)$  at layer  $l_c$ 
15:    if  $|eConn| > M_{\text{max}}$  then ▷ shrink connections of  $e$  (if  $l_c = 0$ , then  $M_{\text{max}} = M_{\text{max}0}$ )
16:       $eNewConn \leftarrow \text{SELECT-NEIGHBORS}(e, eConn, M_{\text{max}}, l_c)$  ▷ Algorithm 5 or 6
17:      set  $neighborhood(e)$  at layer  $l_c$  to  $eNewConn$ 
18:    end if
19:  end for
20:   $ep \leftarrow W$ 
21: end for
22: if  $l > L$  then
23:   set entry point of  $h_{\text{nsw}}$  to  $q$ 
24: end if

```

ANN Layer Search To obtain the approximate ef nearest neighbors of a query element q within a given layer l_c , the algorithm (Algorithm 4) maintains a list W of up to ef closest found elements, initialized with the entry points. At each step, the closest non-evaluated element from the candidate set C is selected, and its neighbors are examined. If a neighbor is closer to q than the furthest element currently in W , or if W contains fewer than ef elements, it is added to both C and W . The process continues until all elements in C have been evaluated.

A key stopping condition is that once the closest candidate in C is further from q than the most distant element in W , the search terminates, as no remaining candidates can improve the result. This mechanism prevents unnecessary distance evaluations and avoids bloating of the search structures [17].

Algorithm 4 SEARCH-LAYER [17]

Require: query element q , entry points ep , number of nearest neighbors ef , layer number l_c **Ensure:** ef nearest neighbors to q

```
1:  $v \leftarrow ep$  ▷ set of visited elements
2:  $C \leftarrow ep$  ▷ set of candidates
3:  $W \leftarrow ep$  ▷ dynamic list of found nearest neighbors
4: while  $|C| > 0$  do
5:    $c \leftarrow$  extract nearest element from  $C$  to  $q$ 
6:    $f \leftarrow$  get furthest neighbor from  $W$  to  $q$ 
7:   if  $distance(c, q) > distance(f, q)$  then
8:     break ▷ all elements in  $W$  evaluated
9:   end if
10:  for each  $e \in neighborhood(c)$  at layer  $l_c$  do ▷ update  $C$  and  $W$ 
11:    if  $e \notin v$  then
12:       $v \leftarrow v \cup \{e\}$ 
13:       $f \leftarrow$  get furthest neighbor from  $W$  to  $q$ 
14:      if  $distance(e, q) < distance(f, q)$  or  $|W| < ef$  then
15:         $C \leftarrow C \cup \{e\}$ 
16:         $W \leftarrow W \cup \{e\}$ 
17:        if  $|W| > ef$  then
18:          remove furthest neighbor from  $W$  to  $q$ 
19:        end if
20:      end if
21:    end if
22:  end for
23: end while
24: return  $W$ 
```

Simple Neighbor Selection A straightforward approach for selecting connections in the proximity graph is to simply select the closest elements to the base element from the candidate set (Algorithm 5). This strategy does not account for the mutual distances between candidates, but only their distances to the base element, which may lead to redundant connections in highly clustered regions of the space [17].

Algorithm 5 SELECT-NEIGHBORS-SIMPLE [17]

Require: base element q , candidate elements C , number of neighbors to return M **Ensure:** M nearest elements to q

```
1: return  $M$  nearest elements from  $C$  to  $q$ 
```

Heuristic Neighbor Selection To address the limitations of the simple strategy, a heuristic procedure is employed for selecting neighbors (Algorithm 6). The heuristic processes the candidates in order of increasing distance to the base element and establishes a connection to a candidate only if its distance to the base element is smaller than its distance to any of the already selected neighbors. In this way, the algorithm promotes diversity of connections by preventing redundant edges in similar directions.

Two optional parameters further refine this method. The first, *extendCandidates*, allows the candidate set to be expanded by including neighbors of the initial candidates; although disabled by default, this option is beneficial in extremely clustered data. The second, *keepPrunedConnections*, enables adding back some of the discarded candidates in order to maintain a fixed number of connections per element [17].

Algorithm 6 SELECT-NEIGHBORS-HEURISTIC [17]

Require: base element q , candidate elements C , number of neighbors to return M , layer number l_c , flag indicating whether or not to extend candidate list *extendCandidates*, flag indicating whether or not to add discarded elements *keepPrunedConnections*

Ensure: M elements selected by the heuristic

```

1:  $R \leftarrow \emptyset$ 
2:  $W \leftarrow C$  ▷ queue for candidates
3: if extendCandidates then ▷ extend candidates by their neighbors
4:   for each  $e \in C$  do
5:     for each  $e_{adj} \in \text{neighborhood}(e)$  at layer  $l_c$  do
6:       if  $e_{adj} \notin W$  then
7:          $W \leftarrow W \cup \{e_{adj}\}$ 
8:       end if
9:     end for
10:  end for
11: end if
12:  $W_d \leftarrow \emptyset$  ▷ queue for discarded candidates
13: while  $|W| > 0$  and  $|R| < M$  do
14:    $e \leftarrow$  extract nearest element from  $W$  to  $q$ 
15:   if  $e$  is closer to  $q$  compared to any element from  $R$  then
16:      $R \leftarrow R \cup \{e\}$ 
17:   else
18:      $W_d \leftarrow W_d \cup \{e\}$ 
19:   end if
20: end while
21: if keepPrunedConnections then ▷ add some of discarded connections from  $W_d$ 
22:   while  $|W_d| > 0$  and  $|R| < M$  do
23:      $R \leftarrow R \cup \{\text{extract nearest element from } W_d \text{ to } q\}$ 
24:   end while
25: end if
26: return  $R$ 

```

k -ANN Search The retrieval of the k approximate nearest neighbors of a query element q in the HNSW structure (Algorithm 7) follows a process analogous to inserting a zero-level element. The key distinction is that the closest neighbors at the ground layer are returned as the search result rather than being used to form new connections.

Starting from the top layer L , the algorithm traverses down through the layers in a greedy manner. At each layer, the search begins from the element closest to the query found in the previous layer and uses $ef = 1$ to quickly move toward the query. Upon reaching the ground layer ($l_c = 0$), a broader greedy search is conducted with the parameter ef to identify the closest elements, from which the k nearest neighbors are returned [17].

Algorithm 7 K-NN-SEARCH [17]

Require: multilayer graph h_{NSW} , query element q , number of nearest neighbors k , size of dynamic candidate list ef

Ensure: k nearest neighbors to q

```

1:  $W \leftarrow \emptyset$  ▷ set of current nearest elements
2:  $ep \leftarrow$  get entry point of  $h_{\text{NSW}}$ 
3:  $L \leftarrow$  level of  $ep$  ▷ top layer in  $h_{\text{NSW}}$ 
4: for  $l_c \leftarrow L \dots 1$  do
5:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef = 1, l_c)$ 
6:    $ep \leftarrow$  get nearest element from  $W$  to  $q$ 
7: end for
8:  $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef, l_c = 0)$ 
9: return  $k$  nearest elements from  $W$  to  $q$ 

```

3.3.4 Analysis of HNSW

Construction Parameters

The main construction parameters of HNSW are m_L , M_{max0} , M , and $ef_{\text{Construction}}$. These parameters directly influence the graph structure, search performance, and memory usage. The analysis follows [17].

m_L Parameter The parameter m_L controls the probability of assigning elements to higher layers, thereby affecting the hierarchy depth and the navigability of the graph. Its influence can be summarized as:

- $m_L = 0$ and $M_{\text{max0}} = M$: produces a directed k -NN graph with power-law complexity.
- $m_L = 0$ and $M_{\text{max0}} = \infty$: produces an NSW graph with polylogarithmic complexity.
- $m_L > 0$: produces a controllable hierarchy graph with logarithmic complexity.

To achieve optimal performance in the controllable hierarchy, the overlap of neighbors across layers (i.e., the percentage of element neighbors shared between layers) should remain small. Reducing m_L decreases this overlap but increases the average number of hops during greedy search, so an optimal tradeoff exists. A practical choice is

$$m_L = \frac{1}{\ln(M)},$$

ensuring on average a single overlapping neighbor between layers.

M_{max0} Parameter The parameter M_{max0} defines the maximum number of connections per element in the ground layer and has a strong influence on search performance, particularly for high-recall queries. Setting $M_{\text{max0}} = M$ leads to a significant performance penalty at high recall. Simulations in [17] suggest that setting $M_{\text{max0}} = 2 \cdot M$ provides good performance, while increasing it further may result in performance degradation and unnecessary memory usage.

***M* Parameter** The parameter M defines the number of neighbors to be selected for each element during insertion. It determines how many bidirectional connections a new element establishes in each layer. A reasonable range for M is 5 to 48. Smaller values generally yield better performance for low-dimensional data or low-recall queries, while larger values are more suitable for high-dimensional data or high-recall searches. Memory usage grows proportionally with M , so it should be chosen carefully.

***efConstruction* Parameter** The parameter *efConstruction* defines the size of the candidate list used during the insertion of a new element. A sufficiently large *efConstruction* ensures that most true nearest neighbors of the element are considered, achieving high recall during index construction. Increasing *efConstruction* can further improve index quality, but at the cost of longer construction times.

Complexity

Search Complexity A theoretical analysis of a single search, following [17], assumes exact Delaunay graphs. Suppose the search has identified the closest element at some layer and then descends to the next one. Since the layers are not correlated with the spatial positions of the data points, there is a fixed probability that the next visited node belongs to the upper layer, given by $p = e^{-m_L}$. However, the search within a layer must terminate before reaching such a node; otherwise, the search on the upper layer would have already stopped at a different element. Consequently, the probability of not reaching the target within s steps is bounded by $e^{-s \cdot m_L}$, and the expected number of steps in a layer is given by the geometric series

$$S = \frac{1}{1 - e^{-m_L}},$$

which is independent of N . If the average degree of a node in the (exact) Delaunay graph is bounded by a constant C , then the expected number of distance evaluations per layer is bounded by a constant $C \cdot S$. Since the expected number of layers in the structure grows as $O(\log N)$ by construction, the overall expected search complexity is

$$O(\log N).$$

In practice, the assumption of having an exact Delaunay graph is violated in HNSW due to the use of an approximate neighbor selection heuristic with a fixed number of connections per element. To avoid getting stuck in local minima at the ground layer, the greedy search algorithm employs a backtracking procedure. For low-dimensional data, the dependence of the required *ef* parameter (which controls the number of hops explored during backtracking) on dataset size saturates as N increases, so the backtracking complexity behaves as an additive term and does not affect the overall $O(\log N)$ search scaling [17].

Construction Complexity The index is built by inserting all elements iteratively [17]. Each insertion consists of a sequence of k -ANN searches across the layers, followed by applying a heuristic with fixed cost for a given *efConstruction*. The average number of layers an element belongs to is a constant depending only on m_L . Therefore, the insertion complexity has the same scaling as the search, and for relatively low-dimensional datasets the overall construction complexity is

$$O(N \cdot \log N).$$

Memory Cost The memory consumption of HNSW index is primarily determined by the storage of graph connections. Each element has up to $M_{\max0}$ connections in the ground layer and up to $M_{\max}$ connections in all other layers. Therefore, the average memory required per element is

$$(M_{\max0} + m_L \cdot M_{\max}) \cdot \text{bytes_per_link}.$$

If the maximum number of elements is limited to approximately four billion, four-byte unsigned integers can be used to store the connections. Typical near-optimal values of M range from 6 to 48, leading to average memory requirements of roughly 60–450 bytes per element, excluding the size of the data itself [17].

* * *

Three prominent approaches for approximate nearest neighbor search have been presented: Locality-Sensitive Hashing (LSH), Approximate Nearest Neighbors Oh Yeah (Annoy), and Hierarchical Navigable Small World (HNSW). LSH emphasizes theoretical guarantees and sub-linear query time through hash-based partitioning of the data space. Annoy focuses on practical efficiency, using tree-based random projections and supporting memory-friendly, multi-process usage. HNSW combines hierarchical graph structures with scale separation, achieving strong empirical performance and logarithmic search complexity. Together, these methods exemplify three distinct paradigms for approximate nearest neighbor search: hash-based, tree-based, and graph-based.

Chapter 4

Vector Databases in Practice

4.1 Systems and Tools

Vector databases provide specialized infrastructure for managing high-dimensional embeddings, enabling efficient similarity search over large collections of unstructured data. They are optimized for real-time queries, large-scale vector operations, and integrated index management, making them a core component in AI-driven applications.

Vector Databases, Libraries, and Plugins Vector search libraries, such as FAISS, are designed to be integrated into the application under development. They provide high-performance ANN search, but scalability becomes an issue as dataset sizes grow. Traditional relational databases or search systems, such as Elasticsearch, often include built-in vector search plugins. However, these are enhancements on top of existing architectures and are therefore limited in functionality and performance. Dedicated vector databases, such as Milvus or Weaviate, are purpose-built to deliver scalable, real-time responses and comprehensive APIs for both vector and structured data management.

4.1.1 FAISS

FAISS (Facebook AI Similarity Search) is an open-source¹ library for ANN search. Its core is implemented in C++ with no external dependencies and is accompanied by a Python interface, allowing it to be used both in standalone scripts and as a building block for larger systems.

FAISS provides multiple indexing techniques, which may include preprocessing, compression, and non-exhaustive search strategies - specifically, inverted file and graph-based methods, such as HNSW.

At its core, FAISS maintains an index to which database vectors are incrementally added. Queries are submitted as vectors, and the library returns nearest neighbors according to a chosen distance metric, most commonly the Euclidean distance. Variants include returning k nearest neighbors, vectors within a specified radius, batch queries, and execution on CPUs or GPUs.

It is important to note that FAISS does not perform feature extraction, manage concurrency, or provide database services such as sharding, transaction management, or query optimization. Its scope is focused on the algorithmic implementation of ANN search [3, 30].

¹<https://github.com/facebookresearch/faiss>

4.1.2 Milvus

Milvus is an open-source² vector database. Written in Go and C++, Milvus leverages CPU and GPU acceleration to enhance vector search performance. Kubernetes-native architecture allows horizontal scaling, real-time streaming updates, and deployment in both multi-node clusters and standalone mode.

It stores vectors alongside scalar metadata and enables efficient k -NN queries, hybrid search, and metadata filtering. The system supports multiple vector index types, including HNSW, IVF, FLAT, SCANN, and DiskANN, with quantization-based variations.

Milvus also supports sparse vectors for full-text search and hybrid retrieval, combining dense and sparse embeddings within the same collection. Data security is enforced through mandatory authentication, TLS encryption, and role-based access control.

The Milvus ecosystem integrates with various AI development frameworks and provides utilities for embedding transformation, search reranking, administration, monitoring, and data pipeline integration [4, 31].

4.1.3 Weaviate

Weaviate is an open-source³, cloud-native vector database that stores both objects and vectors, enabling semantic search at scale. It combines vector similarity search with keyword filtering, retrieval-augmented generation (RAG), and reranking in a single query interface, supporting applications such as RAG systems, semantic and image search, recommendation engines, chat-bots, and content classification.

Weaviate allows vectors to be either automatically generated at import using integrated models (e.g., OpenAI, Cohere, HuggingFace) or directly imported as pre-computed embeddings. Its production-ready features include multi-tenancy, replication, fine-grained role-based access control (RBAC), and horizontal scaling.

The system supports advanced hybrid search by combining semantic, keyword (BM25), and image search, and integrates generative RAG and reranking capabilities. Performance optimizations, such as vector compression and multi-vector encoding, reduce memory usage while maintaining fast, millisecond-scale retrieval over billions of vectors [5].

4.2 Practical Applications

This section demonstrates the practical applications of vector databases through experiments with FAISS, Milvus, and Weaviate. The aim is to illustrate how embeddings can be stored, indexed, and queried.

Experiments were conducted using Google Colab to provide a consistent and accessible execution environment. FAISS, being a standalone library, was integrated directly into the Colab notebook without additional setup. For Milvus and Weaviate, cloud-based sandbox environments were employed. Each service required the creation of a cluster, followed by retrieval of the connection parameters necessary to interact with the respective database. Once the environments were established, all subsequent operations, including collection creation, embeddings insertion, indexing, and nearest neighbors search, were executed through the code examples presented in this section.

²<https://github.com/milvus-io/milvus>

³<https://github.com/weaviate/weaviate>

4.2.1 Image Search

Image search involves retrieving visually or semantically similar images given a query image. Traditional approaches, relying on exact pixel comparisons or handcrafted features, often fail to capture higher-level semantic content and do not scale efficiently to large datasets. Representing images as high-dimensional embeddings allows the use of nearest neighbor techniques to measure similarity in a feature space that encodes semantic relationships. Vector databases facilitate efficient storage, indexing, and retrieval of these embeddings, enabling nearest neighbor search even over large-scale datasets. This makes the approach a practical and scalable solution for content-based image retrieval tasks.

Dataset and Embeddings Demonstrations were conducted on a subset of the CIFAR-10 dataset, consisting of 60,000 color images of size 32×32 pixels, evenly distributed across ten object classes. Each image in the subset is labeled according to its class (`labels`). Feature embeddings were extracted using a pretrained ResNet-18 model, yielding high-dimensional representations suitable for nearest neighbor search (`embeddings`). Detailed dataset preparation and embeddings extraction steps are provided in Appendix A.1.

Demonstrations The examples provided below illustrate the use of vector management systems and tools for nearest neighbor search on image embeddings. The demonstration uses pre-extracted variables including the embedding dimension (`d`), the query vector (`query_vector`, corresponding to the first image in the subset), and the number of nearest neighbors (`k`, set to 10). The label of the query image is 6, which occurs in 207 images within the subset. Definitions of these variables are provided in Appendix A.2. Below each code example, the output shows the k returned IDs and their corresponding labels.

FAISS HNSW

The first example demonstrates the use of the FAISS library, employing the HNSW algorithm for approximate nearest neighbor search. The code performs k -NN search using pre-extracted embeddings, as shown in Listing 4.1.

Listing 4.1: k -NN search using FAISS HNSW approach.

```
1 import numpy as np
2 import faiss
3
4 index_hnsw = faiss.IndexHNSWFlat(d, 32) # M=32
5 index_hnsw.hnsw.efConstruction = 40
6 index_hnsw.add(embeddings)
7
8 index_hnsw.hnsw.efSearch = 50
9 distances_hnsw, indices_hnsw = index_hnsw.search(query_vector.reshape(
    (1, -1), k)
10
11 print("Top-k indices:", indices_hnsw[0].tolist())
12 print("Top-k labels:", labels[indices_hnsw[0]].tolist())
```

```
Top-k indices: [0, 1002, 1484, 228, 1393, 314, 1635, 1596, 27, 1202]
Top-k labels: [6, 3, 6, 6, 6, 3, 6, 6, 5, 3]
```

Milvus HNSW

The second example demonstrates the use of Milvus, a cloud-based vector database. Connection setup details are provided in Appendix A.3. The code performs k -NN search using pre-extracted embeddings, as shown in Listing 4.2.

Listing 4.2: k -NN search using Milvus HNSW approach.

```
1 import numpy as np
2 from pymilvus import FieldSchema, DataType, CollectionSchema,
   Collection, list_collections
3
4 collection_name = "CIFAR10"
5
6 # Drop the collection if it already exists to ensure a clean start
7 if collection_name in list_collections():
8     Collection(collection_name).drop()
9
10 # Define and create the collection schema
11 # 'index' serves as the primary key, 'embedding' stores the image
   vectors
12 collection = Collection(
13     collection_name,
14     CollectionSchema([
15         FieldSchema(name="index", dtype=DataType.INT64, is_primary=
           True),
16         FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR,
           dim=d)
17     ])
18 )
19
20 # Insert embeddings with their indices and flush to persist data
21 collection.insert([np.arange(len(embeddings)), embeddings])
22 collection.flush()
23
24 # Create an HNSW index on the embedding field
25 index_params = {"index_type": "HNSW", "metric_type": "L2", "params":
   {"M": 32, "efConstruction": 40}}
26 collection.create_index(field_name="embedding", index_params=
   index_params)
27
28 # Load the collection into memory to enable search
29 collection.load()
30
31 # Define search parameters and perform top-k nearest neighbor search
32 search_params = {"metric_type": "L2", "params": {"ef": 50}}
33 results = collection.search([query_vector], "embedding",
   search_params, limit=k)
34
35 top_ids = [res.index for res in results[0]]
36 print("Top-k ids:", top_ids)
37 print("Top-k labels:", labels[top_ids].tolist())
```

Top-k ids: [0, 1002, 1484, 228, 1393, 314, 1635, 1596, 27, 865]

Top-k labels: [6, 3, 6, 6, 6, 3, 6, 6, 5, 3]

Weaviate HNSW

The third example demonstrates the use of Weaviate, a cloud-based vector database. Connection setup details are provided in Appendix A.4. The code performs k -NN search using pre-extracted embeddings, as shown in Listing 4.3.

Listing 4.3: k -NN search using Weaviate HNSW approach.

```
1 import weaviate.classes as wvc
2
3 collection_name = "CIFAR10"
4
5 # Drop the collection if it already exists to ensure a clean start
6 if collection_name in client.collections.list_all():
7     client.collections.delete(collection_name)
8
9 # Create a new collection with a single INT property 'index'
10 # vector_config=self_provided() indicates that embeddings are
    precomputed
11 collection = client.collections.create(
12     name=collection_name,
13     properties=[wvc.config.Property(name="index", data_type=wvc.
        config.DataType.INT)],
14     vector_config=wvc.config.Configure.Vectors.self_provided()
15 )
16
17 # Insert embeddings with their unique indices
18 collection.data.insert_many([
19     wvc.data.DataObject(properties={"index": i}, vector=e)
20     for i, e in enumerate(embeddings)
21 ])
22
23 # Perform top-k nearest neighbor search
24 response = collection.query.near_vector(near_vector=query_vector,
    limit=k)
25
26 top_ids = [res.properties["index"] for res in response.objects]
27 print("Top-k ids:", top_ids)
28 print("Top-k labels:", labels[top_ids].tolist())
```

Top-k ids: [0, 1819, 1635, 314, 1002, 1628, 265, 1246, 1646, 1393]

Top-k labels: [6, 6, 6, 3, 3, 6, 9, 6, 6, 6]

4.2.2 Recommendation Systems

Recommendation systems aim to predict user preferences and suggest relevant items, such as products, movies, or songs. High-dimensional embeddings can represent both users and items in a shared feature space, capturing latent characteristics and similarities. Vector databases enable efficient nearest neighbor search to identify items most similar to a given user profile or query item, supporting real-time personalized recommendations even for very large catalogs. Approximate nearest neighbor search allows scaling to millions of items without sacrificing responsiveness.

4.2.3 Document Retrieval and Analysis

Document retrieval and analysis involve searching, clustering, and organizing large collections of unstructured text data. Embeddings derived from language models capture semantic relationships between documents beyond keyword matching. Storing these embeddings in vector databases allows for fast similarity search, enabling tasks such as semantic search, document clustering, and question-answering over large corpora. This approach provides both efficiency and scalability, crucial for handling modern text-intensive applications.

* * *

Vector databases provide purpose-built infrastructure for storing and querying embeddings at scale, enabling a wide range of AI applications. Libraries like FAISS offer algorithmic building blocks, while dedicated databases such as Milvus and Weaviate provide fully-managed solutions. Together, these technologies illustrate the spectrum of tools available for managing unstructured data in practical scenarios.

The Google Colab notebook⁴ containing the demonstrations presented in this chapter is publicly available for viewing.

⁴https://colab.research.google.com/drive/1JHNvJkmdoVusyAvnL5KEk_dYkhFJgKAa?usp=sharing

Conclusion

This thesis has examined the theoretical foundations and practical applications of vector databases, with a particular focus on approximate nearest neighbor (ANN) search in high-dimensional spaces. Through detailed analysis of ANN paradigms, including hash-based methods (LSH), tree-based methods (Annoy), and graph-based methods (HNSW), the study clarified their operational principles, advantages, and limitations, providing insights that complement existing literature.

The practical investigation highlighted contemporary vector management systems such as FAISS, Milvus, and Weaviate. By generating embeddings from a representative dataset, constructing indices, and executing similarity searches, the thesis demonstrated how these systems can be effectively leveraged in real-world scenarios. This hands-on component validated theoretical concepts and provided a clear workflow for implementing similarity search in high-dimensional spaces.

Achieved Objectives of the Thesis

The primary objectives set at the outset have been successfully met:

- Established a comprehensive theoretical foundation, covering vector representations, similarity measures, and nearest neighbor search principles.
- Analyzed approximate nearest neighbor algorithms, including hash-based (LSH), tree-based (Annoy), and graph-based (HNSW) methods, highlighting their characteristics, advantages, and limitations.
- Demonstrated the practical application of vector databases through FAISS, Milvus, and Weaviate, including embeddings storage, index construction, and similarity-based querying, with experiments that bridged theory and practice on real datasets.

Future Directions

Building on the work presented in this thesis, future research could focus on enhancing performance and scalability through techniques such as multithreading, CPU-level optimizations, and low-level programming, which were beyond the scope of this study. Other promising directions include the development of hybrid ANN algorithms, large-scale distributed vector database architectures, and tighter integration with AI pipelines to improve retrieval efficiency and versatility.

Appendices

Appendix A

Code Listings

A.1 Dataset and Embeddings

Listing A.1: Dataset preparation and embeddings extraction.

```
1 import numpy as np
2 import torch, torchvision
3 import torchvision.transforms as transforms
4 from torchvision.models import resnet18, ResNet18_Weights
5
6 # Select device: GPU if available, otherwise CPU
7 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
8
9 # Define image transformations for ResNet input
10 transform = transforms.Compose([
11     transforms.Resize((224, 224)), # ResNet standard input size
12     transforms.ToTensor(),
13     transforms.Normalize(
14         mean=[0.485, 0.456, 0.406], # ImageNet mean
15         std=[0.229, 0.224, 0.225] # ImageNet std
16     )
17 ])
18
19 # Load CIFAR-10 dataset (subset for demonstration)
20 dataset = torchvision.datasets.CIFAR10(
21     root='./data',
22     train=True,
23     download=True,
24     transform=transform
25 )
26
27 subset_size = 2000
28 subset_indices = list(range(subset_size))
29 subset = torch.utils.data.Subset(dataset, subset_indices)
30 loader = torch.utils.data.DataLoader(subset, batch_size=64, shuffle=
    False)
31
32 # Initialize pretrained ResNet18 model
33 model = resnet18(weights=ResNet18_Weights.DEFAULT)
```

```
34 model.fc = torch.nn.Identity() # Remove classification layer to get
    embeddings
35 model = model.to(device)
36 model.eval()
37
38 # Compute embeddings for each image in the subset
39 embeddings = []
40 labels = []
41
42 with torch.no_grad():
43     for images, lbls in loader:
44         images = images.to(device)
45         emb = model(images) # Extract feature vectors
46         embeddings.append(emb.cpu().numpy())
47         labels.extend(lbls.numpy())
48
49 # Convert lists to NumPy arrays for downstream usage
50 embeddings = np.vstack(embeddings)
51 labels = np.array(labels)
52
53 print("Embeddings shape:", embeddings.shape)
54 print("Labels shape:", labels.shape)
```

A.2 Variables Extraction

Listing A.2: Extraction of variables used in demonstration examples.

```
1 # Embedding dimension
2 d = embeddings.shape[1]
3
4 # Query vector: first sample in dataset
5 query_vector = embeddings[0]
6
7 # Number of nearest neighbors to retrieve
8 k = 10
9
10 print(f"Query label: {labels[0]}")
11 print(f"Number of samples with label {labels[0]}: {np.sum(labels ==
    labels[0])}")
```

A.3 Milvus Connection

Listing A.3: Connection to Milvus cluster from Google Colab.

```
1 from pymilvus import connections
2
3 connections.connect(
4     alias="default",
5     uri="YOUR_URI",
6     token="YOUR_TOKEN"
7 )
```

A.4 Weaviate Connection

Listing A.4: Connection to Weaviate cluster from Google Colab.

```
1 import weaviate
2 from weaviate.classes.init import Auth
3
4 client = weaviate.connect_to_weaviate_cloud(
5     cluster_url="YOUR_CLUSTER_URL",
6     auth_credentials=Auth.api_key("YOUR_API_KEY")
7 )
```

Bibliography

- [1] N. Ukey, Z. Yang, B. Li, G. Zhang, Y. Hu, and W. Zhang, “Survey on Exact kNN Queries over High-Dimensional Data Space,” *Sensors*, vol. 23, no. 2, p. 629, January 2023.
- [2] M. Grohe, “word2vec, node2vec, graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data,” *arXiv preprint arXiv:2003.12590*, March 2020.
- [3] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The Faiss Library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [4] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, and X. Xu, “Milvus: A Purpose-Built Vector Data Management System,” in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2614–2627.
- [5] Weaviate B.V. Weaviate. Accessed: September 22, 2025. [Online]. Available: <https://github.com/weaviate/weaviate>
- [6] A. Gionis, P. Indyk, and R. Motwani, “Similarity Search in High Dimensions via Hashing,” in *Proceedings of the 25th International Conference on Very Large Data Bases*, September 1999, pp. 518–529.
- [7] A. Guttman, “R-Trees: A Dynamic Index Structure for Spatial Searching,” *ACM SIGMOD Record*, vol. 14, no. 2, pp. 47–57, June 1984.
- [8] M. Skrodzki, “The k-d Tree Data Structure and a Proof for Neighborhood Computation in Expected Logarithmic Time,” *arXiv preprint arXiv:1903.04936*, March 2019.
- [9] M. Dolatshah, A. Hadian, and B. M. Bidgoli, “Ball*-Tree: Efficient Spatial Indexing for Constrained Nearest-Neighbor Search in Metric Spaces,” *arXiv preprint arXiv:1511.00628*, November 2015.
- [10] A. W.-C. Fu, P. M.-S. Chan, Y.-L. Cheung, and Y. S. Moon, “Dynamic vp-Tree Indexing for n-Nearest Neighbor Search Given Pair-Wise Distances,” *The VLDB Journal*, vol. 9, pp. 154–173, January 2000.
- [11] N. Kouriroukidis and G. Evangelidis, “The Effects of Dimensionality Curse in High Dimensional kNN Search,” in *2011 15th Panhellenic Conference on Informatics*, October 2011.
- [12] P. Indyk and R. Motwani, “Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality,” in *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, May 1998, pp. 604–613.

- [13] M. Patella and P. Ciaccia, “Approximate Similarity Search: A Multi-Faceted Problem,” *Journal of Discrete Algorithms*, vol. 7, no. 1, pp. 36–48, March 2009.
- [14] E. Bernhardsson. Annoy (Approximate Nearest Neighbors Oh Yeah). Accessed: September 18, 2025. [Online]. Available: <https://github.com/spotify/annoy>
- [15] W. Li, Y. Zhang, Y. Sun, W. Wang, W. Zhang, and X. Lin, “Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement,” *arXiv preprint arXiv:1610.02495*, October 2016.
- [16] E. Bernhardsson. Approximate Nearest Neighbor Methods and Vector Models - NYC ML Meetup. Accessed: September 19, 2025. [Online]. Available: <https://github.com/erikbern/ann-presentation>
- [17] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *arXiv preprint arXiv:1603.09320*, March 2016.
- [18] —, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, April 2020.
- [19] E. Bernhardsson. Benchmarking Nearest Neighbors. Accessed: August 28, 2025. [Online]. Available: <https://github.com/erikbern/ann-benchmarks>
- [20] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, “Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs,” *Information Systems*, vol. 45, pp. 61–68, 2014.
- [21] M. Boguna, D. Krioukov, and K. Claffy, “Navigability of Complex Networks,” *arXiv preprint arXiv:0709.0303*, September 2007.
- [22] S. Milgram, “The Small World Problem,” *Psychology Today*, vol. 2, no. 1, pp. 60–67, 1967.
- [23] J. Guare, *Six Degrees of Separation: A Play*. Vintage Books, November 1990, play.
- [24] J. M. Kleinberg, “The Small-World Phenomenon: An Algorithmic Perspective,” in *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, May 2000, pp. 163–170.
- [25] —, “Navigation in a Small World,” *Nature*, vol. 406, p. 845, August 2000.
- [26] J. S. B. Mitchell and W. Mulzer, “Proximity Algorithms,” in *Handbook of Discrete and Computational Geometry*, 3rd ed., J. E. Goodman, J. O’Rourke, and C. D. Tóth, Eds. Boca Raton, FL: CRC Press, 2017, ch. 48.
- [27] F. B. Moghaddam, S. Banihashemi, R. Bakhshoodeh, J. M. F. Mir, and I. N. Delgado, “Biomimicry Green Facade: Integrating Nature into Building Facades for Enhanced Building Envelope Efficiency,” *SSRN Electronic Journal*, July 2023.
- [28] F. Aurenhammer, “Voronoi Diagrams - A Survey of a Fundamental Geometric Data Structure,” *ACM Computing Surveys*, vol. 23, no. 3, pp. 345–405, September 1991.

- [29] G. Navarro, “Searching in Metric Spaces by Spatial Approximation,” *The VLDB Journal*, vol. 11, pp. 28–46, 2002.
- [30] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [31] R. Guo, X. Luan, L. Xiang, X. Yan, X. Yi, J. Luo, Q. Cheng, W. Xu, J. Luo, and F. Liu, “Manu: A Cloud Native Vector Database Management System,” *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3548–3561, 2022.